



COMP 4300

Intro to Game Programming

Lecture 13

Game Camera / Views

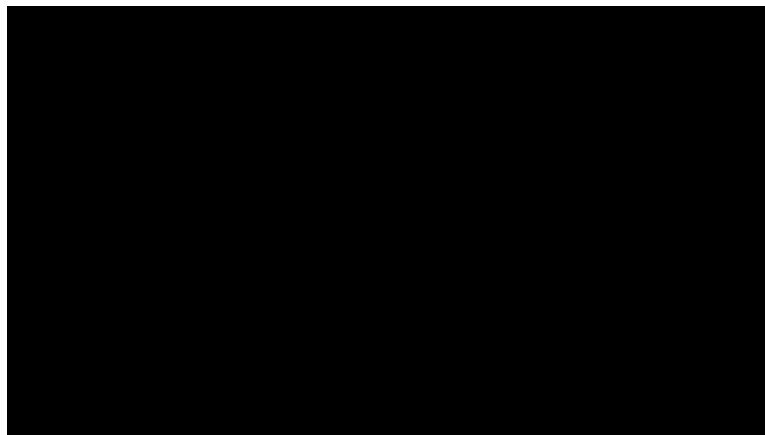
Visualizing Games

- Video game worlds are often much bigger than we can display on-screen all at once
- Like the real world, we can only display a sub-section of the world at a given time
- The part of the game world that we can currently see is called a **View**

View / Camera

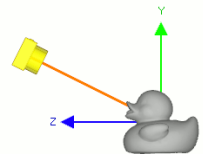
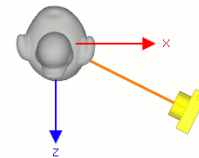
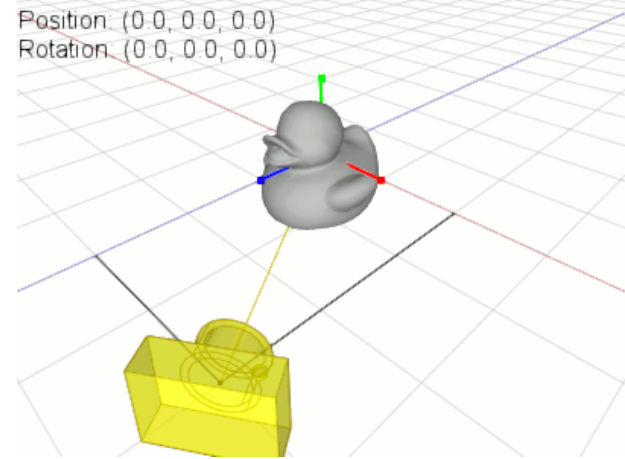
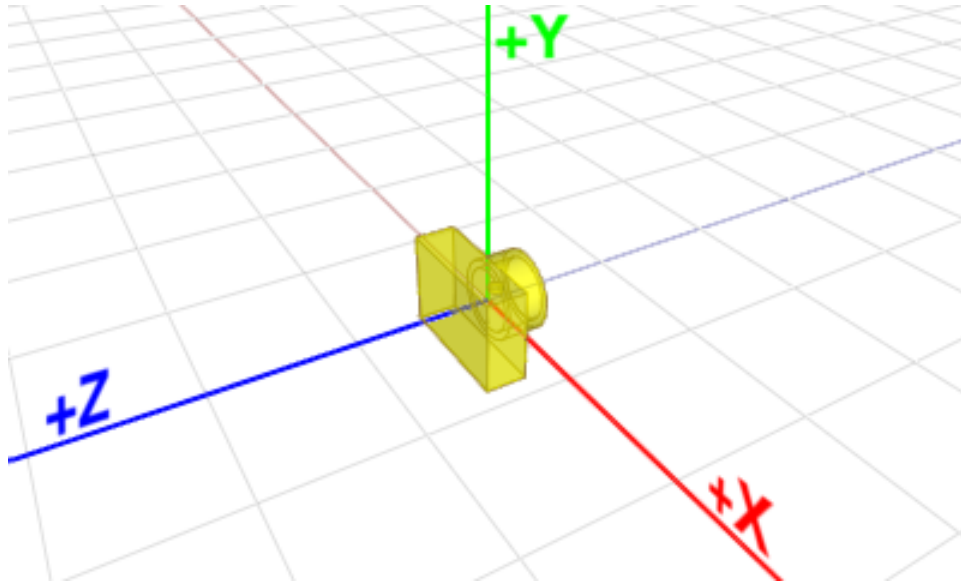
- To mathematically describe our desired view of the game world, we often use the concept of an in-game **Camera**
- Intuitively, we define the location of a camera in the world, and where it points
- Similar to real photography, this will determine the final view that we see

“The Challenge of Cameras”

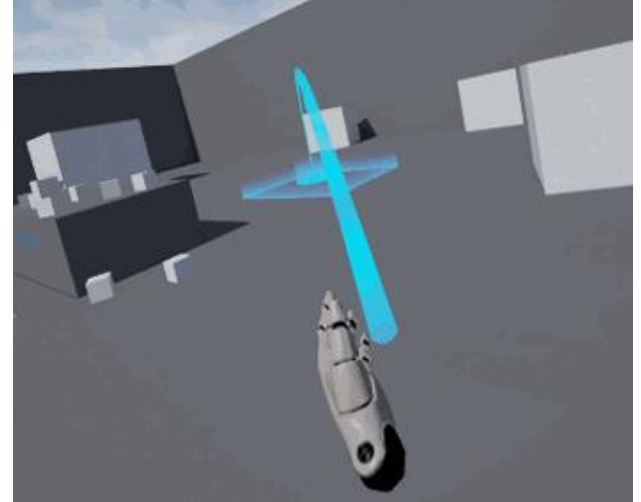
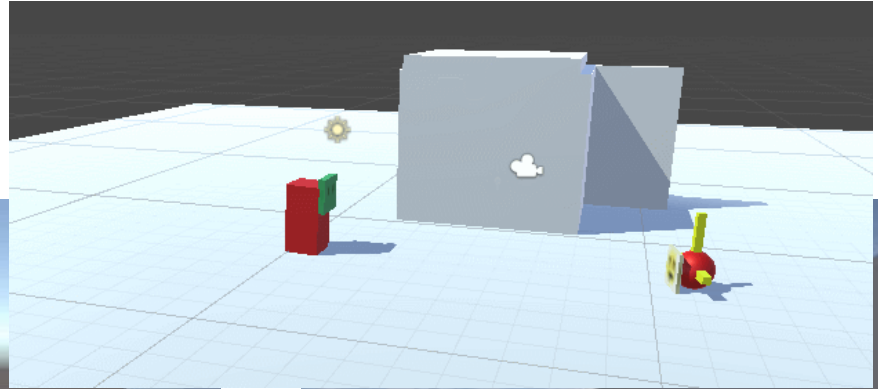
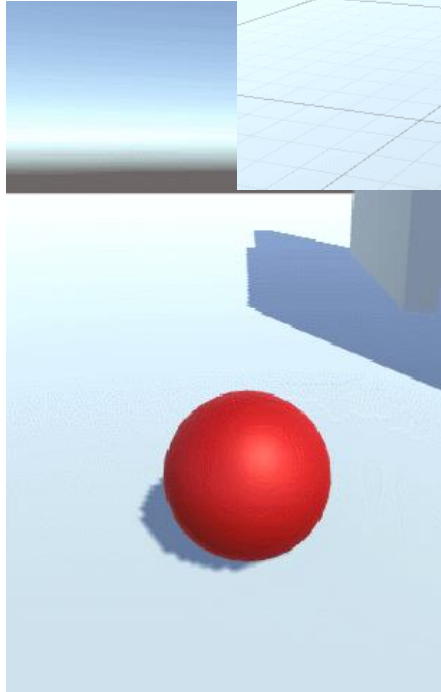
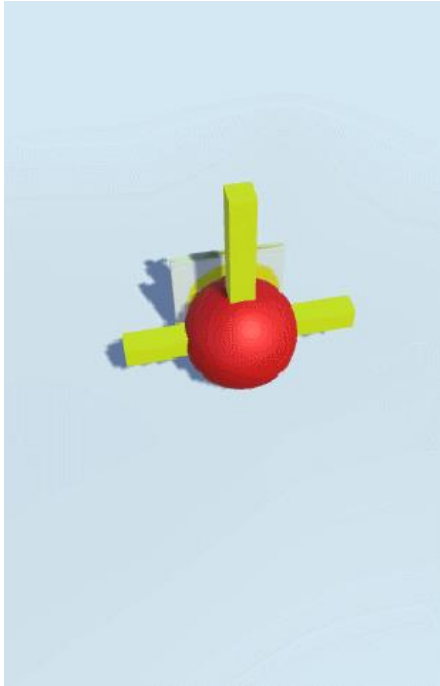


<https://www.youtube.com/watch?v=bHdi5Ar8GXw>

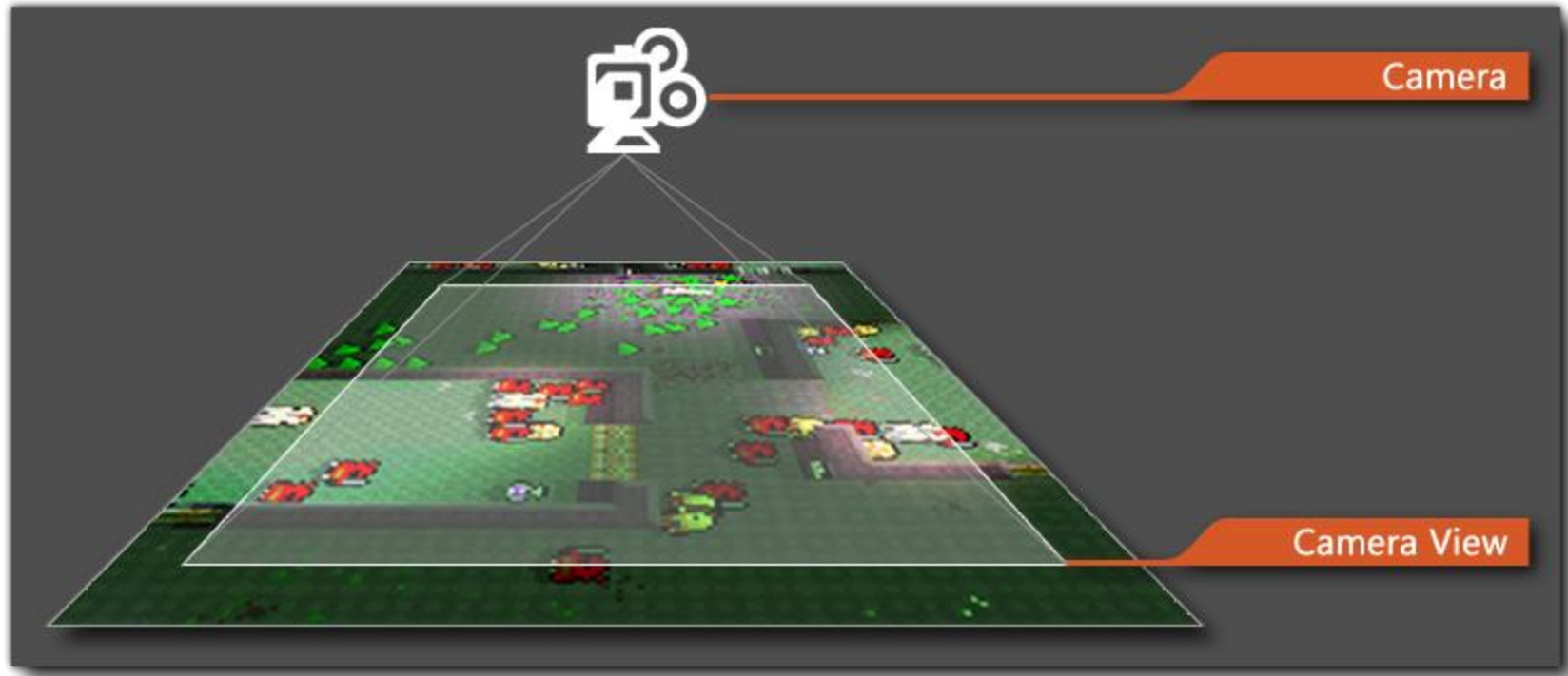
3D Game Camera



Camera Views



2D Game Camera



Camera Uses



2D Camera Types



<https://www.youtube.com/watch?v=I9G6MNHfV7M>

SFML Views

- A 'View' is a window into another 'world'
- An `sf::View` is a rectangle representing a 2D **camera** that you can use to specify which pixels of a game will be drawn to the window
- Changing a window's view does not alter the underlying entity/sprite positions, it simply changes where in the window they are drawn
- By changing the view, we can change what part of our game world is shown to the player

```
sf::RenderWindow window;  
sf::View view;
```

<https://www.sfml-dev.org/tutorials/2.5/graphics-view.php>

```
// Initialize the view to a rectangle located at (100, 100) and with a size of 400x200  
view.reset(sf::FloatRect(100, 100, 400, 200));
```

```
// Rotate it by 45 degrees  
view.rotate(45);
```

```
// Set its target viewport to be half of the window  
view.setViewport(sf::FloatRect(0.f, 0.f, 0.5f, 1.f));
```

```
// Apply it  
window.setView(view);
```

```
// Render stuff  
window.draw(someSprite);
```

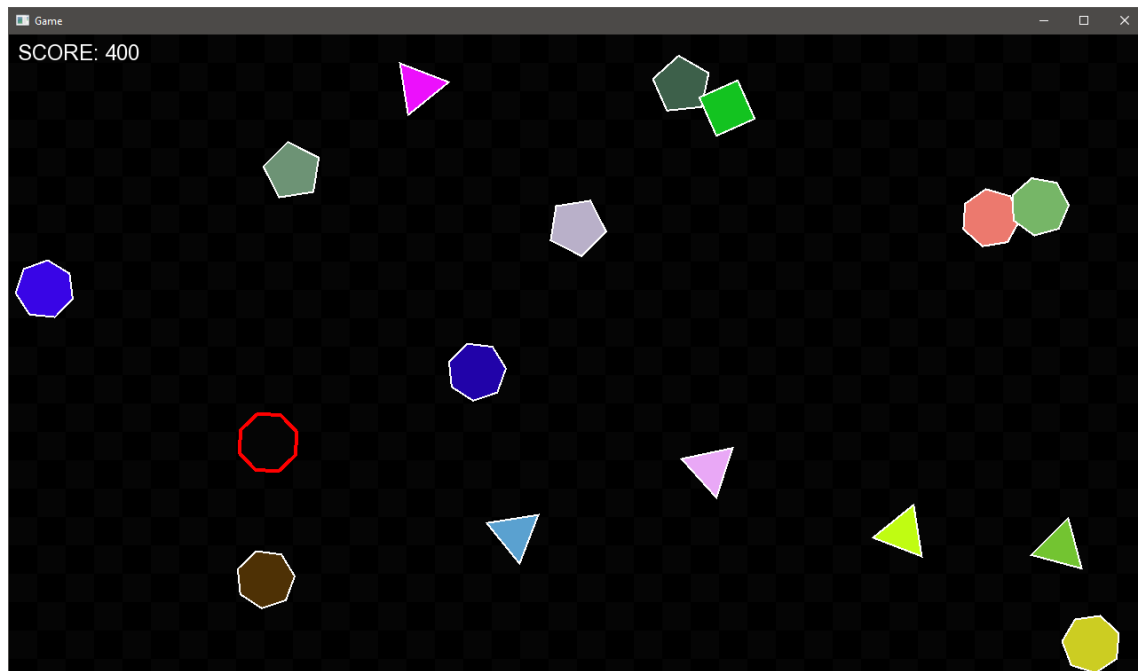
```
// Set the default view back  
window.setView(window.getDefaultView());
```

```
// Render stuff not affected by the view  
window.draw(someText);
```

“The Theory and Practice of Cameras in SideScrollers”

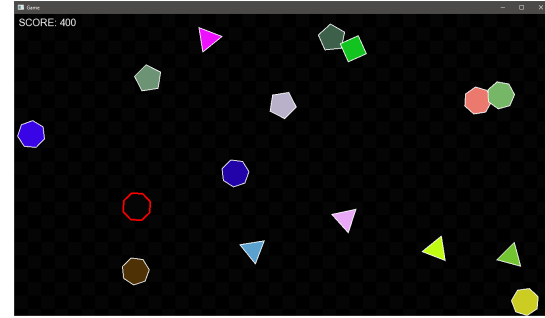
https://www.gamasutra.com/blogs/ItayKeren/20150511/243083/Scroll_Back_The_Theory_and_Practice_of_Cameras_in_SideScrollers.php

Default View



Default View

- SFML default view
- Window constructed with this view
- Top-Left is (0,0), Bottom-Right is (w,h)
- View doesn't move, so entire game world must fit into the window dimensions
- Works well for certain arcade-style games or board games, but very limited



Position Lock

- Camera centered on player's position
- Best in games with smooth motion overall
- Best when character is small in comparison to the map view



Camera Smoothing

- Any of these techniques can employ different camera movement techniques
- Auto lock: camera instantly on target
- Smoothing: camera glides to target
- Choose which fits your game best





Horizontal Scroll View

- Camera follows player in x-axis
- Can lead to jerky movements L/R
- Useful when view does not go higher than 'one screen'



Direction Scrolling



Box/Trap View

- Define a box around the player
- View only moves if player leaves box
- Useful for games with jerky back and forth movement





Position Snapping

- Follow character in one axis (x-axis)
- Snap position if the character position in other axis changes
- Smoother camera movement overall



Platform Snapping

- Like position snapping, but occurs on level geometry instead
- Only snap the view when a player lands on a platform



Platform Snapping

- Like position snapping, but occurs on level geometry instead
- Only snap the view when a player lands on a platform

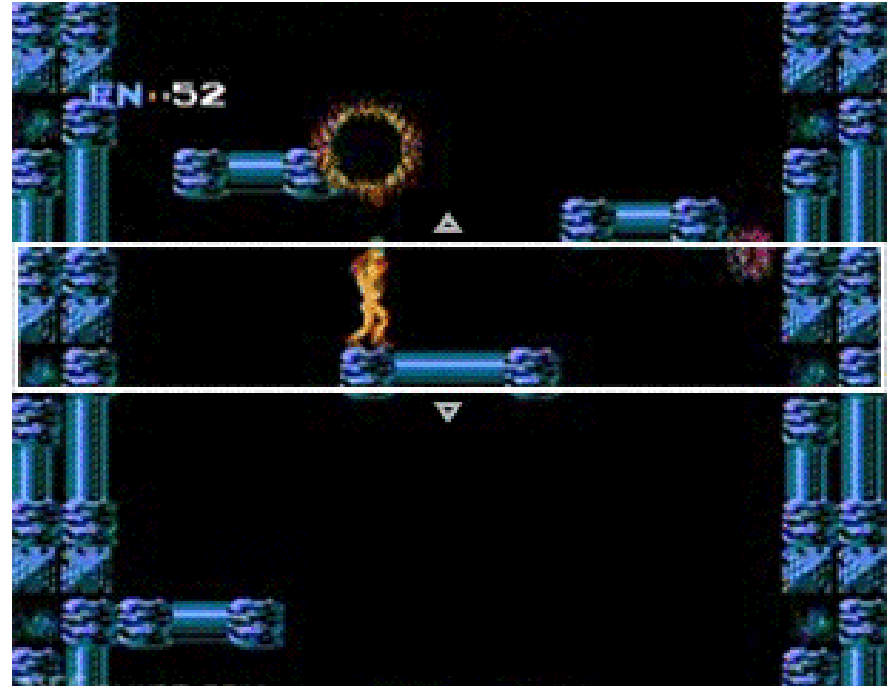


Camera Zones

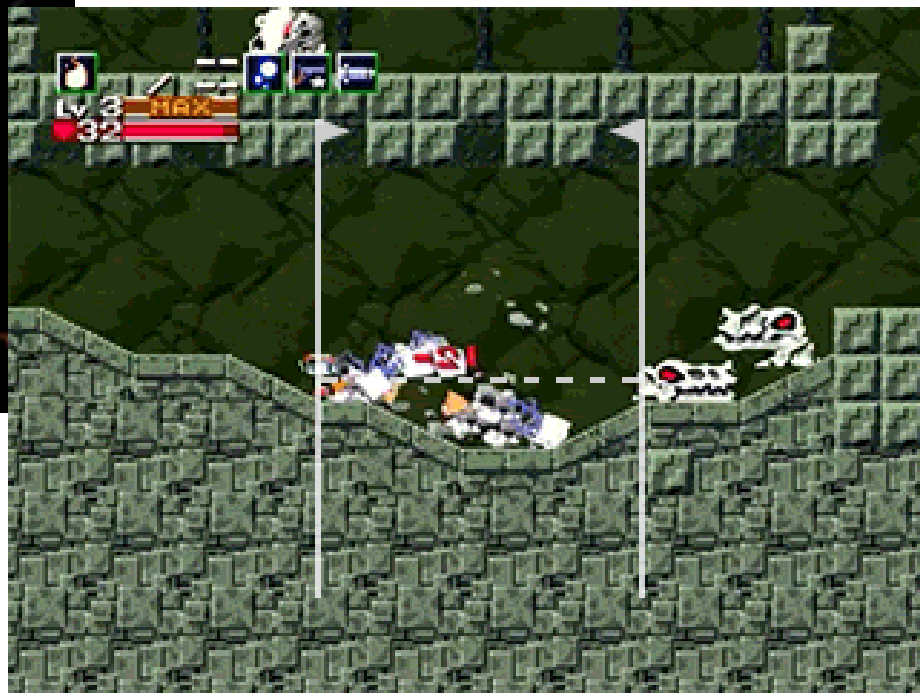
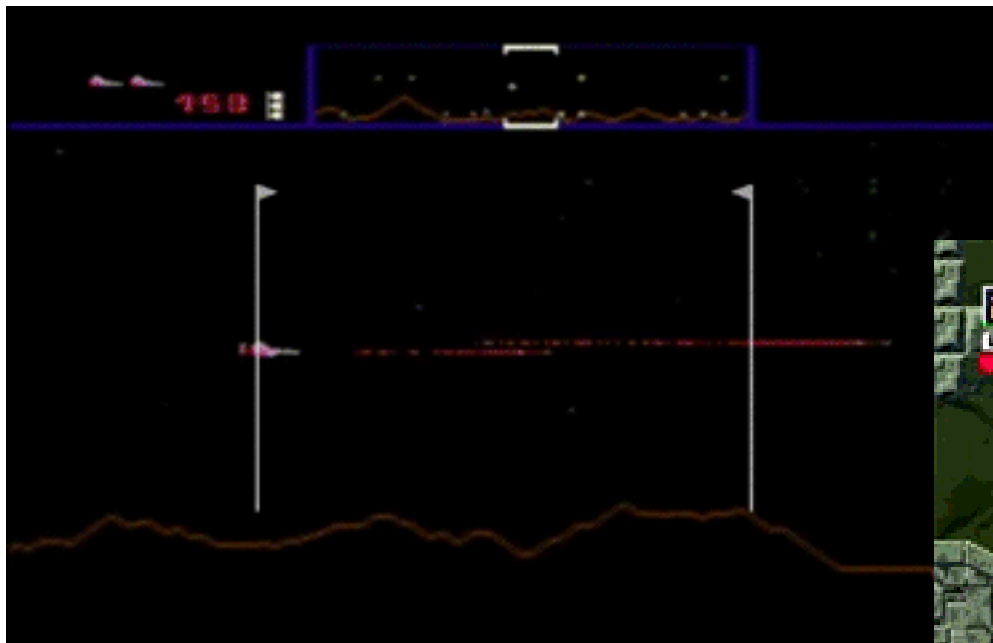
- Define zones for camera movement
- Can speed up / stop camera based on direction of movement and location with zone



Camera Zones







Camera Zooming/Focus

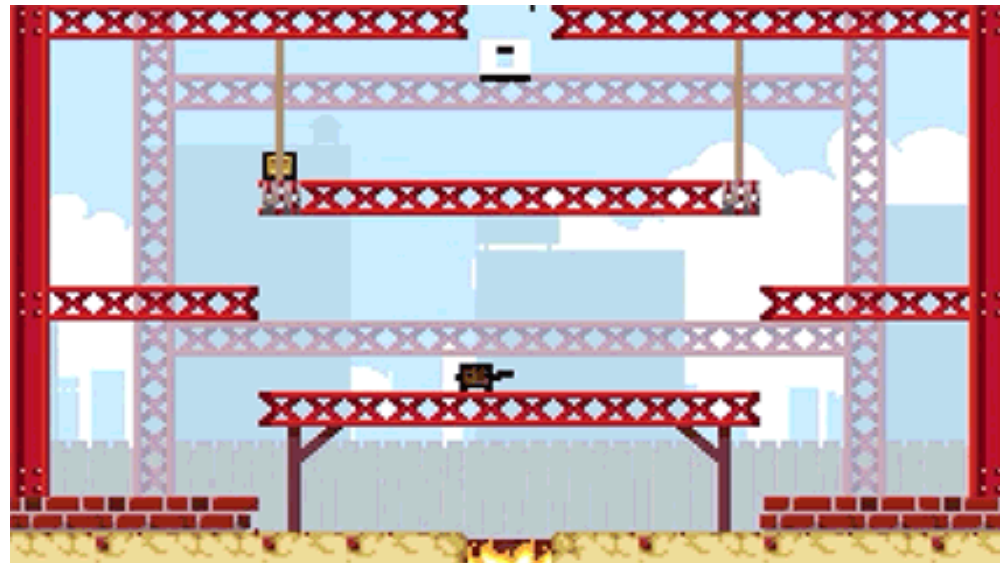


Camera Window

- Camera window can grow and shrink as players get closer or further apart
- Very useful for games light fighting games where both players must be visible at all times



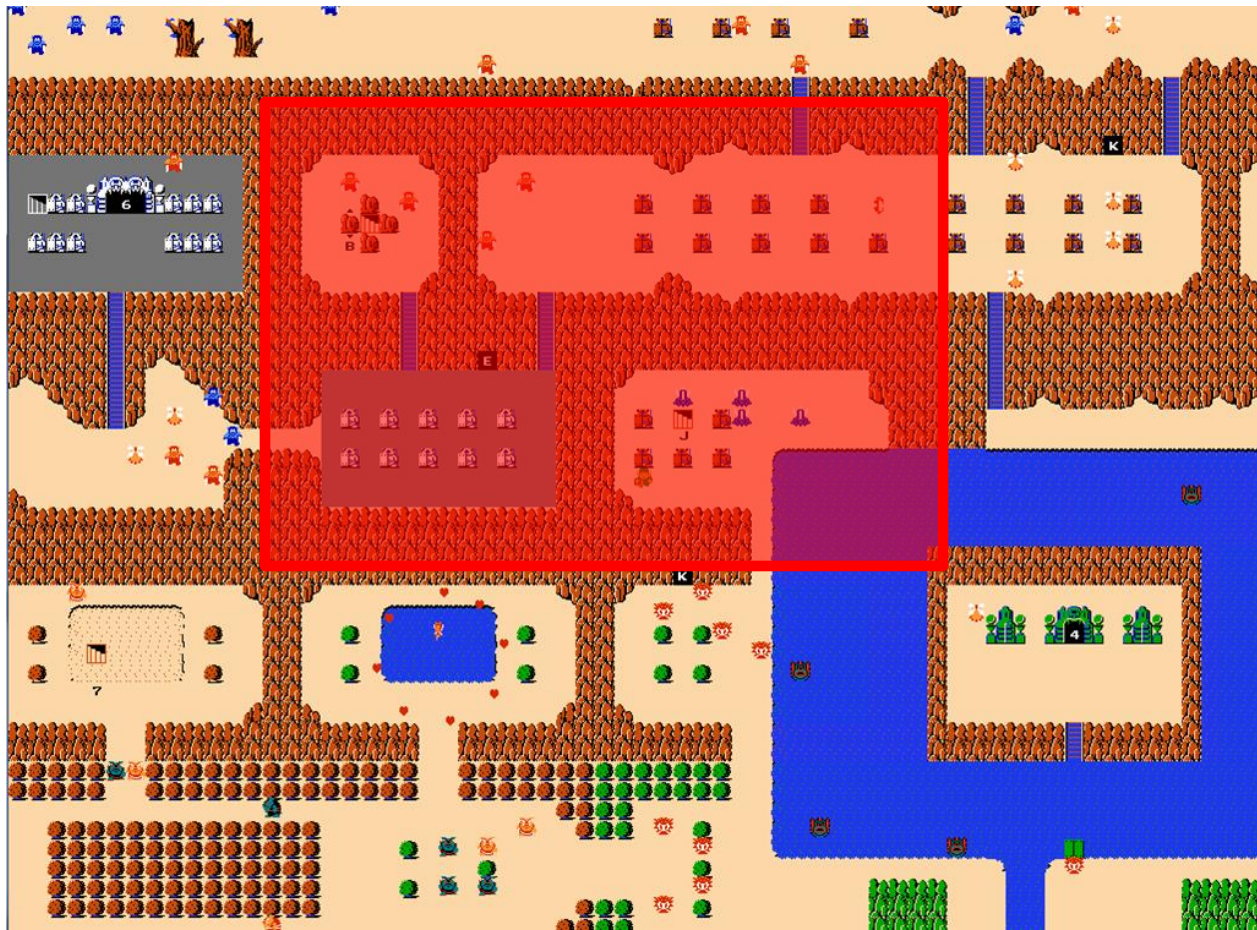
Camera Shake





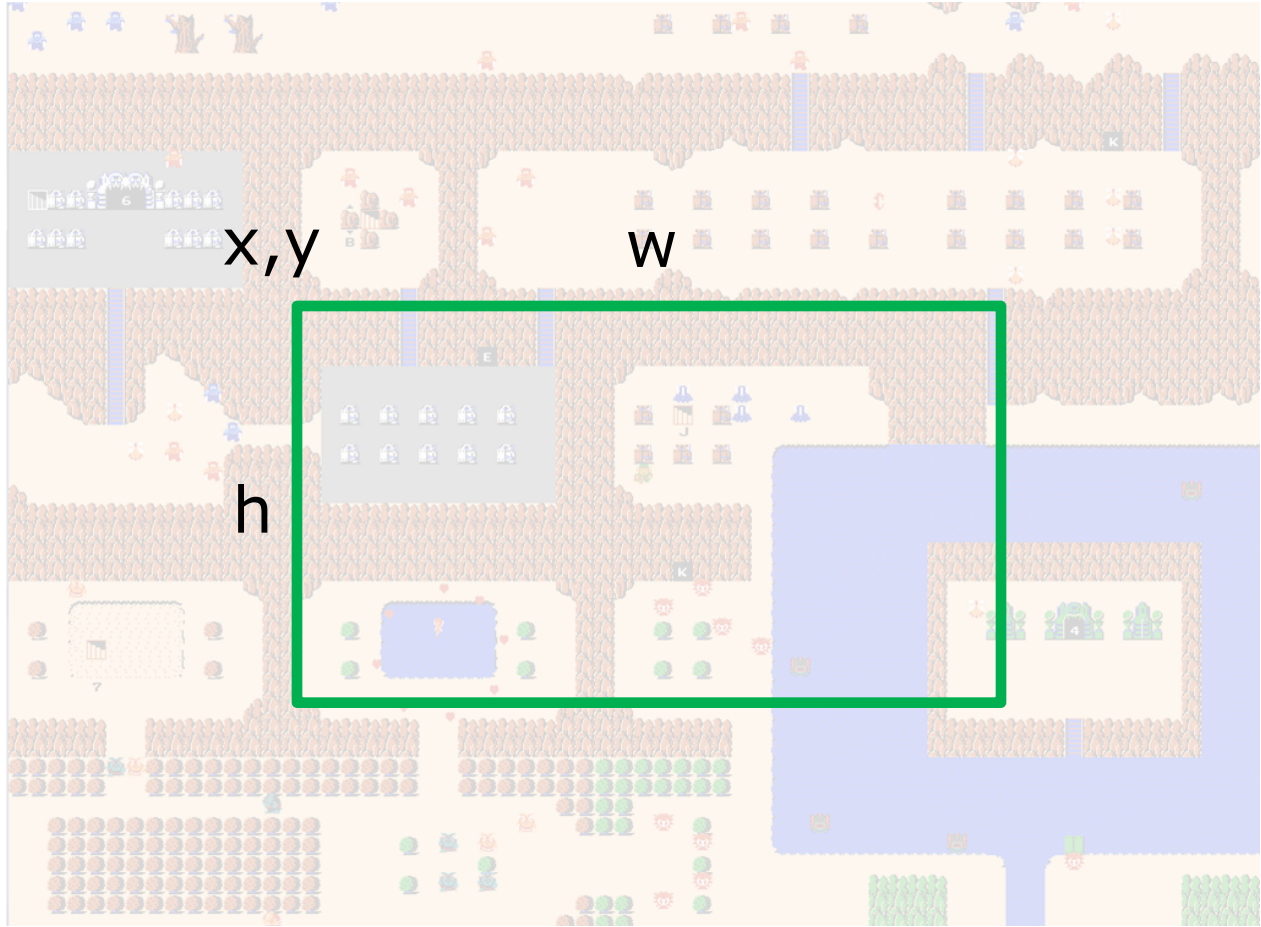






Defining a View

- A view can be constructed in two ways
 - `sf::View(sf::FloatRect(leftX, topY, width, height))`
 - `sf::View(sf::Vector2f(cx, cy), sf::Vector2f(w, h))`
- This means that the same resulting view can be accomplished in multiple ways, for example:
 - `sf::View(x, y, w, h)`
 - `sf::View((x + w/2, y + h/2), (w, h))`
- Using the top-left or center may be more convenient based on the game you're making



x, y

w

h



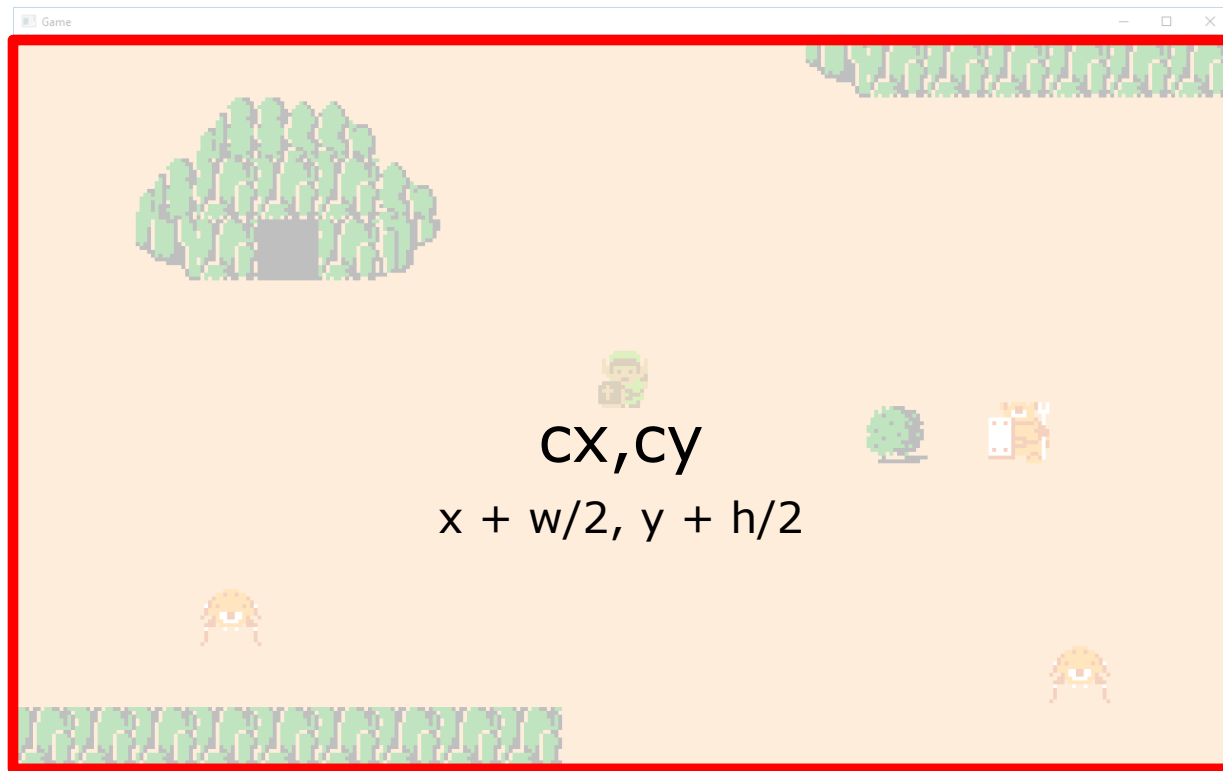
x, y

w

h

cx, cy

$x + w/2, y + h/2$



Setting a View

- Views can be set after window constructed
 - `view.reset(sf::FloatRect(x,y,w,h))`
 - `view.setCenter(sf::Vector2f(cx,cy))`
 - `view.setSize(sf::Vector2f(w,h))` (keeps center)
- Views can be moved
 - `view.move(dx, dy)`
 - View rectangle pos becomes $(x + dx, y + dy)$
- View can be zoomed toward center
 - `view.zoom(z)` $z < 1$ = close, $z > 1$ = far

View Rotation

- Views can be rotated by a given angle
 - `view.rotate(angle)`
- Controls same, only the view of the game world will change



Viewports

- Views define which part of the game world will be drawn to the window
- Viewports define where on the window the view of the world will be drawn
 - Example: draw only to left half of screen
- By default, the contents of the view occupy the entire window

Defining a Viewport

- Viewport is not defined in pixels, but instead as a ratio of the window size
- The viewport is set as part of a view
 - `View.setViewport(sf::FloatRect(x,y,w,h))`
 - (x,y) = ratio of window for top left of render
 - (w,h) = ratio of window for size of render

```
// define a centered viewport, with half the size of the window  
view.setViewport(sf::FloatRect(0.25f, 0.25, 0.5f, 0.5f));
```



Viewport Usage

- Viewports can be used to create interesting / useful views for gameplay
- For example, 2-player split-screen
 - `p1view.setViewport(sf::FloatRect(0, 0, 0.5, 1));`
 - `p2view.setViewport(sf::FloatRect(0.5, 0, 0.5, 1));`

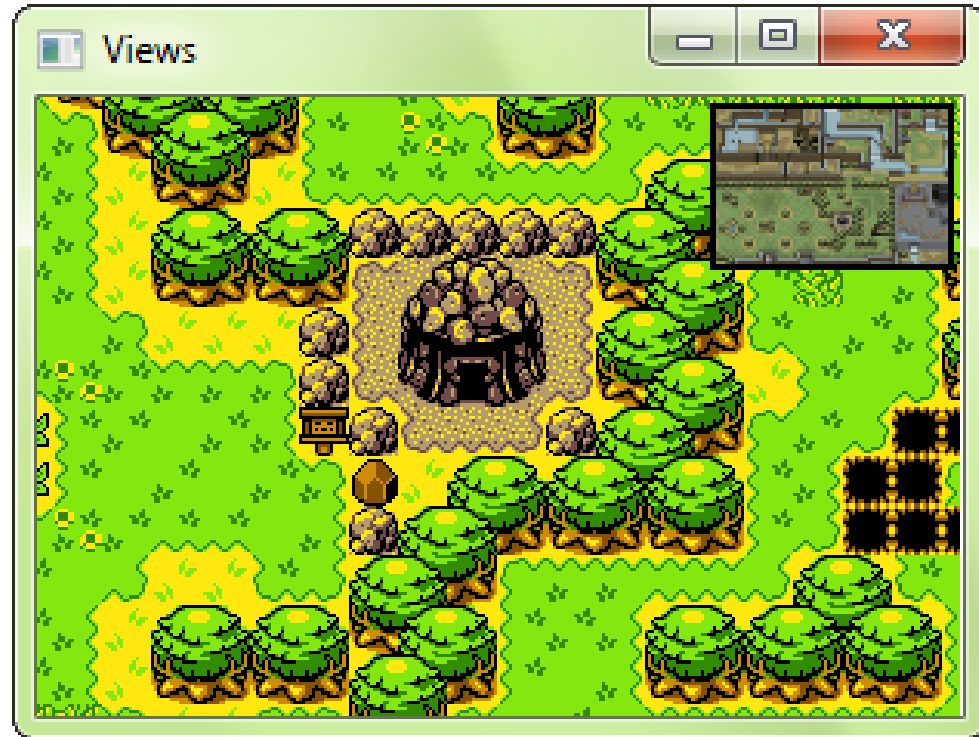
Viewport Split Screen



Viewport Usage

- Viewports can be used to create interesting / useful views for gameplay
- For example, 2-player split-screen
 - `p1view.setViewport(sf::FloatRect(0, 0, 0.5, 1));`
 - `p2view.setViewport(sf::FloatRect(0.5, 0, 0.5, 1));`
- Or you can create a mini-map
 - `gameView.setViewport(sf::FloatRect(0, 0, 1, 1));`
 - `miniView.setViewport(sf::FloatRect(0.75, 0, 0.25, 0.25));`

Viewport Mini Map



Viewport Usage

- Viewports can be used to create interesting / useful views for gameplay
- For example, 2-player split-screen
 - `p1view.setViewport(sf::FloatRect(0, 0, 0.5, 1));`
 - `p2view.setViewport(sf::FloatRect(0.5, 0, 0.5, 1));`
- In-game mini-map
- Rear-view mirror
- Enemy perspective

Implementing Camera Types

- There are two main ways that we could implement different camera types
 1. Implement camera types inside a camera ECS system with an Enum
 2. Implement a Camera class that is passed a reference to the GameEngine

Camera System

```
1. namespace CameraTypes
2. {
3.     FollowX, FollowXY, Box, Zelda, Default;
4. }

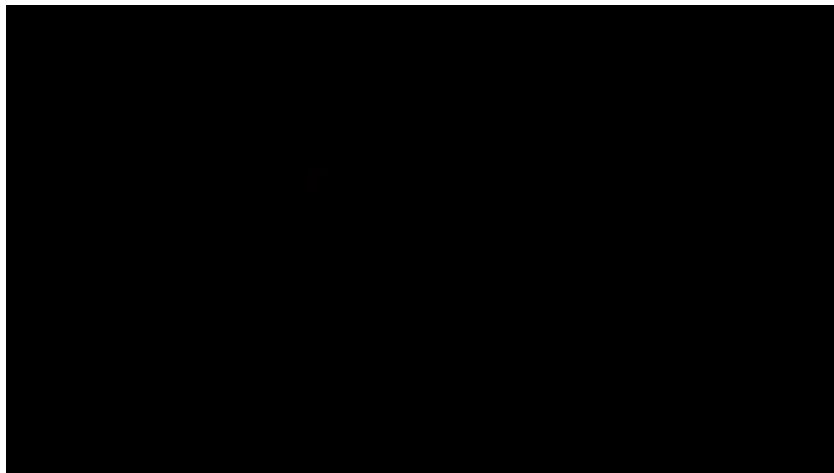
5. void sCamera()
6. {
7.     if (m_cameraType == CameraTypes::FollowXY) { ... }
8.     if (m_cameraType == CameraTypes::Box)      { ... }
9.     if (m_cameraType == CameraTypes::Zelda)    { ... }
10. }
```

Camera Class

```
1. class Camera
2. {
3.     GameEngine * m_engine;
4.     size_t      m_cameraType;
5. public:
6.     Camera(GameEngine * engine, size_t type);
7.     void update() { // do view logic }
8. }

9. m_camera.update()
```

2D Effects: Parallax



<https://www.youtube.com/watch?v=z9tBce8eFqE>